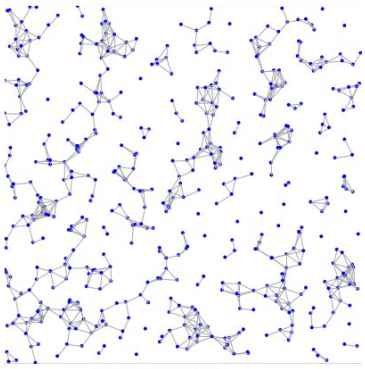
TP – Parcours dans les graphes

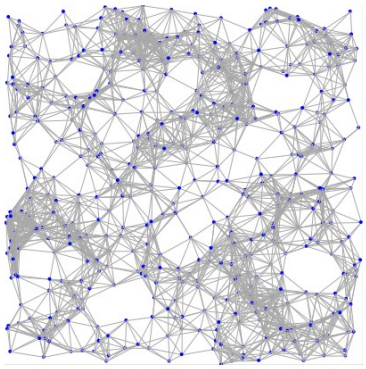
L'objectif premier du TP est d'implémenter la structure de graphe, orienté ou non, à partir de la représentation par tableau de liste de successeurs. Dans un deuxième temps, les parcours en largeur et en profondeur seront implémentés.

Pour la mise en œuvre des listes de successeurs et des parcours, vous disposez des structures de liste (`List`), de file (`Queue`) et de pile (`Stack`) fournies dans les répertoires portant ces noms.

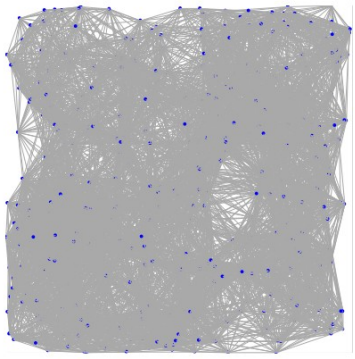
Dans ce TP, les graphes seront construits à partir de points tirés au hasard dans [0,1]x[0,1]. Les sommets du graphe seront les points tirés au hasard. Une arête non orientée {A,B} avec A=(x,y) et B=(x',y') sera présente dans le graphe si la distance entre A et B est plus petite qu'un seuil σ fixé dans [0,1]. Voici trois exemples de graphes non orientés ayant 500 sommets obtenus avec différente valeurs de sigma.



Sigma=0.05



sigma = 0.1



sigma = 0.2

Si le graphe est orienté, une arête (A,B) (de A vers B) avec A=(x,y) et B=(x',y') sera présente dans le graphe si la distance entre A et B est plus petite qu'un seuil σ fixé dans [0,1] et si y<y'. Dans les graphiques précédents, cela revient à orienter les arêtes du haut de l'image vers le bas.

Génération et représentation des graphes par tableau de listes d'adjacence

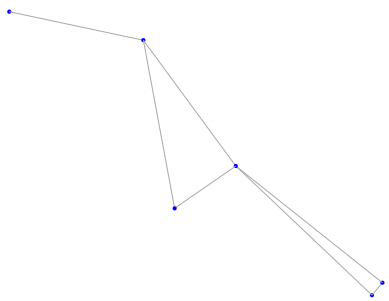
Conserver les coordonnées des sommets et le sigma suffit pour réaliser tous les algorithmes car cela revient à manipuler la matrice d'adjacence (tester si une arête existe entre deux sommets se fait par un calcul simple de distance). Néanmoins, les algorithmes de parcours sont plus efficaces avec la représentation par tableau de listes d'adjacence, surtout si le graphe est peu dense.

La structure de graphe est définie comme suit dans le fichier `graph.h` avec une structure de liste classique de type `List` (voir code fourni).

```
typedef struct Graph {
    int numberVertices;   /** The number of vertices in the graph. */
    List *array;     /** Array of adjacency lists. A null pointer means an empty list*/

    double *xCoordinates;  /** array of the x coordinates of the points of each vertex */
    double *yCoordinates;  /** array of the y coordinates of the points of each vertex */
    double sigma; /** maximal distance between two neighboring vertices */

    int *parents; /** array to store the parent of each node for the covering tree. */


    int *topological_ordering; /** array to store a topological orderingof the graph. */
    double *earliest_start; /** array to store the earliest start of each node. */
    double *latest_start; /** array to store the latest start of each node. */

} Graph;
```

Le nombre de sommets est donnés par le champs `numberVertices`. Les coordonnées des points sont stockées dans les deux tableaux `xCoordinates` et `yCoordinates`. Le champs `sigma` correspond au paramètre sigma. Le tableau `parents` servira uniquement à construire les arbre couvrants à partir des parcours en profondeur et largeur. Toutes ses valeurs seront initialisées à -1. Les 3 autres tableaux seront utilisés dans d'autres Tps sur les graphes.

Pour le graphe ci-contre ayant 6 sommets et généré avec sigma =0.5, nous obtenons la structure suivante :



```
numberVertices : 6
sigma = 0.5
xCoordinates = [0.91,0.93,0.04,0.36,0.58,0.44]
yCoordinates = [0.71,0.68,0.03,0.1,0.4,0.5]
array[0] : 4→1→NULL            array[4]: 5->3->1->0->NULL
array[1] : 4→0→NULL            array[5] : 4->3->NULL
array[2] : 3→NULL
array[3] : 5→4→2→NULL
parents=[-1,-1,-1,-1,-1,-1]
```

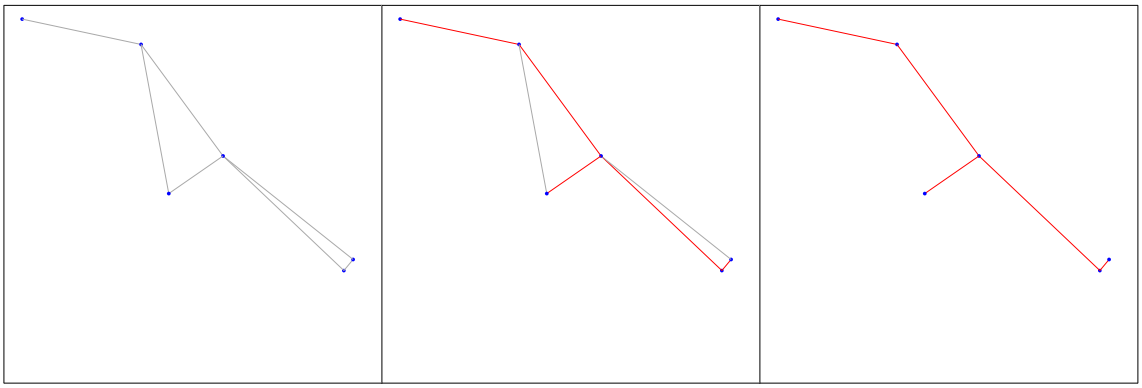
Travail à réaliser :

- Implémentez les fonctions [+] `addEdgeInGraph`, `createGraph` et `printConsoleGraphe`. Les intitulés et les descriptions des fonctions sont suffisamment claires pour ne pas les décrire ici. L'affichage d'un graphe dans la console pourra ressembler à l'image ci-contre (attention, cela ne correspond pas au graphe précédent).

```
---------------
Print graph:
Vertex 0: (0.050319,0.219413)
Vertex 1: (0.807597,0.782765)
Vertex 2: (0.221314,0.461621)
Vertex 3: (0.436210,0.321816)
Vertex 4: (0.258311,0.330447)
Vertex 5: (0.600169,0.790168)
Vertex 0: 4 -> 2 -> NULL
Vertex 1: 5 -> NULL
Vertex 2: 4 -> 3 -> 0 -> NULL
Vertex 3: 4 -> 2 -> NULL
Vertex 4: 3 -> 2 -> 0 -> NULL
Vertex 5: 1 -> NULL
parents: -1 -1 3 4 0 -1
Topological ordering:
[-1 -1 -1 -1 -1 -1 ]
Print start dates:
vertex 0 : [earliest start date=-1.00, latest start date=-1.00 ]
vertex 1 : [earliest start date=-1.00, latest start date=-1.00 ]
vertex 2 : [earliest start date=-1.00, latest start date=-1.00 ]
vertex 3 : [earliest start date=-1.00, latest start date=-1.00 ]
vertex 4 : [earliest start date=-1.00, latest start date=-1.00 ]
vertex 5 : [earliest start date=-1.00, latest start date=-1.00 ]

---------------
```

Une fonction `drawGraph` est fournie qui permet de sauvegarder dans une image de type `svg` un graphe passé en paramètre. La fonction peut sauvegarder le graphe (paramètre `type=0`), la forêt couvrante stockée dans le tableau `parents` et calculée à partir de parcours (paramètre `type=1`), le graphe et la forêt couvrante superposés (paramètre `type=2`) et l'ordre topologique ainsi que les dates de départ des prochains Tps (`type=3`). Il est aussi possible d'indiquer si le graphe est orienté ou non.

Voici trois exemples d'images obtenues (type= 0,1,2) pour des graphes non orientés.



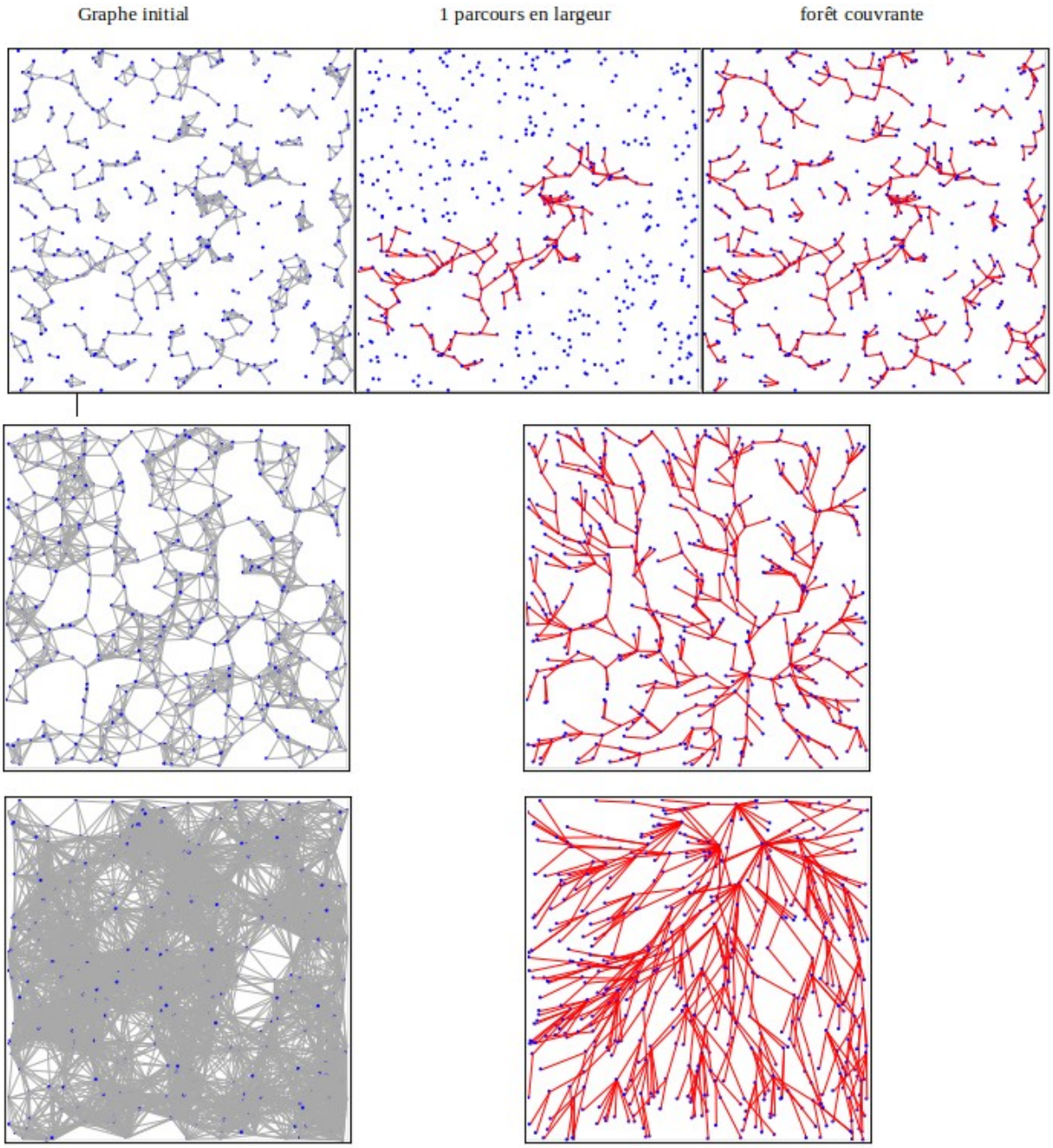
Graphe initial | graphe+forêt obtenue avec un BFS | forêt obtenue avec BFS

Lorsque qu'une recherche en profondeur ou en largeur est réalisée à partir d'un sommet, le parcours va visiter tous les sommets de la composante connexe du sommet initial. Pour construire une forêt couvrante, il faut donc lancer autant de parcours qu'il n'y a de composantes connexes.

Travail à réaliser :

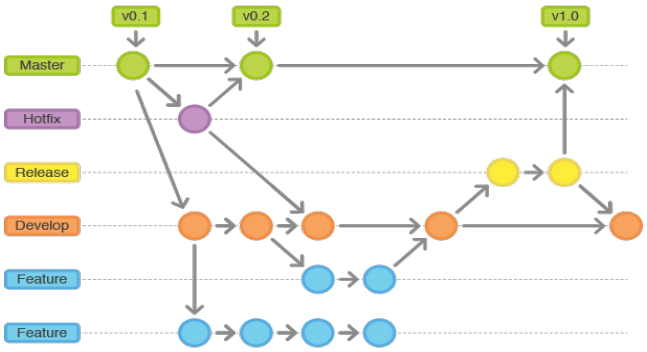
- Implémentez les fonctions [++] `graphDFS` et [++] `graphBFS` qui réalisent respectivement un parcours en profondeur et en largeur sur un graphe donné à partir d'un sommet lui aussi donné. Stockez la structure sous-jacente de forêt couvrante dans le champs `parents`.
- Implémentez la fonction [+] `numberOfComponents` qui calcule le nombre de composantes connexes d'un graphe **non orienté** à partir de plusieurs parcours.
- Pour toutes les fonctions exceptée `printConsoleGraphe,` vous implémenterez des tests unitaires. Ces tests seront à implémenter dans le fichier `testgraph.c` qui ne contient pour le moment qu'un exemple d'utilisation des fonctions.

Voici des exemples de forêts obtenues avec des graphes de 500 sommets pour différentes valeurs de sigma (et des parcours en largeur).

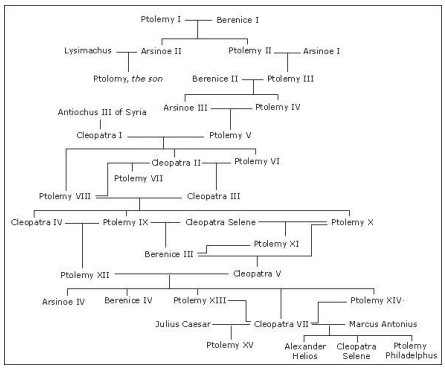


TP – Graphes orientés acycliques

Un graphe orienté est dit acyclique s'il ne contient pas de cycle. Intuitivement, un DAG (Directed Acyclic Graph = graphe orienté acyclique) est composé de sommets « sources » sans arête entrante, de sommets « puits » sans arête sortante et de sommets internes qui permettent de faire le chemin entre les sommets « sources » et les sommets « puits ». Les DAG sont des objets omniprésents dans plusieurs domaines comme la généalogie, la science des données, la planification ou encore l'informatique. Les images ci-dessous (trouvées sur internet) représentent les DAG d'un dépôt Git ou le DAG d'une généalogie.



https://stackoverflow.com/questions/40873964/github-network-graph-not-displaying-correctly



https://en.wikipedia.org/wiki/Directed_acyclic_graph

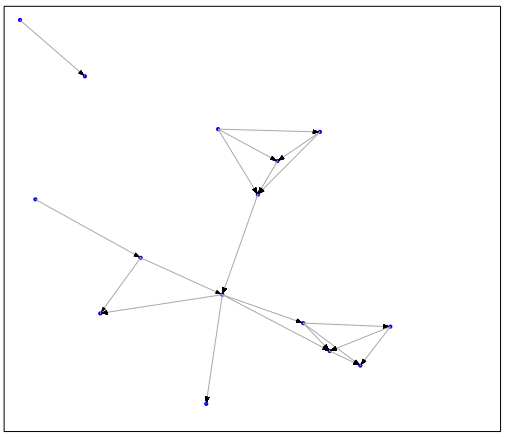
L'objectif de ce TP est de résoudre plusieurs problèmes classiques sur les DAG (complétez le fichier source `topo.c` dans le répertoire `dag`). Vous vous appuierez sur la structure de graphe du précédent TP (`Graph`).

Générer des DAG

On peut générer un DAG à partir d'un ensemble de points dans $[0,1]^2$. Il existe une arête (P_1,P_2) du point $P_1(x_1,y_1)$ vers le point $P_2(x_2,y_2)$ si

1. la distance entre P_1 et P_2 est plus petite qu'un paramètre σ donné
2. et $y_1<y_2$.

La figure ci-contre a été obtenue de cette manière et présente un DAG avec 3 source et 4 puits.



Travail à réaliser : Normalement, générer un DAG a été réalisé dans le précédent TP sur les graphe. Si nécessaire,vérifiez et adaptez.

Ordonner les sommets

Avec le cours, vous disposez de tous les éléments théoriques pour résoudre le problème suivant.

Problème : *étant donné un DAG à n sommets, construire une liste ordonnée de sommets $(x_1,x_2,\ldots,x_n)$ telle que pour tout couple de sommets (x_i,x_j), si x_i est un prédécesseur x_j, alors $i<j$.*

Une telle liste est appelée un ordre topologique du DAG. Une conséquence directe de l'ordre topologique est que le premier sommet de l'ordre est une source alors que le dernier est un puits.

Travail à réaliser : écrivez une fonction *[++]* `topologicalSort(Graph graph)` qui résout le problème et qui stockera la solution dans le tableau `topological_ordering`. Faites un affichage console de la solution. Implémentez aussi des tests unitaires dans le fichier `testtopo.c`.

Attention pour résoudre ce problème, il est explicitement demandé de ne pas utiliser les coordonnées des points (un simple tri suffirait). Il faut s'appuyer sur la représentation par listes d'adjacence.

Dates au plus tôt

On se place ici dans la cadre d'un DAG pondéré *G=(V,E,w)* avec V l'ensemble des sommets, E l'ensemble des arêtes et *w* la fonction de pondération. Dans ce TP, la pondération d'une arête sera naturellement donnée par la distance euclidienne entre les deux sommets qui composent l'arête.

Ce type de DAG modélise par exemple un ensemble de tâches à réaliser avec des contraintes de précédence et un temps d'exécution pour chaque tâche.

Définition : *la date au plus tôt d'un sommet source (sans arête entrante) est fixée à 0. La date au plus tôt d'un sommet s non source (avec des arêtes entrantes) est donnée par :*

$$dtot(s) = max(dtot(s') + w(s',s); s' \text{ prédécesseur de } s)$$

Si chaque sommet représente des travaux à réaliser, la date au plus tôt d'un sommet est la date optimale du début des travaux correspondant au sommet.

Travail à réaliser : écrivez une fonction [++] `computeEarliestStartDates(Graph graph)` qui résout le problème et qui stocke la solution dans le tableau `earliest_start`. Faites des tests unitaires dans le fichier `testtopo.c.`

Dates au plus tard (optionnel)

On se place dans le même cadre que la partie précédente.

Définition : *la date au plus tard d'un sommet puits (sans arête sortante) est fixée au maximum des dates au plot tôt des sommets puits. La date au plus tard d'un sommet s non puits (avec des arêtes sortantes) est donnée par :*

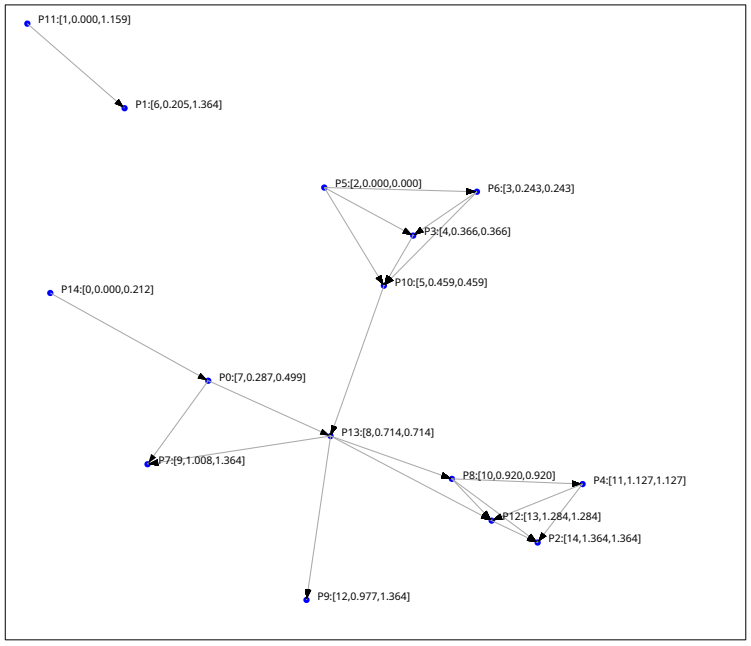
$$dtard(s) = min(dtard(s') - w(s,s'); s' \text{ successeur de } s)$$

Si chaque sommet représente des travaux à réaliser, la date au plus tard d'un sommet est la date maximale à laquelle les travaux correspondant au sommet doivent commencer pour ne pas prendre de retard.

Travail à réaliser : écrivez une fonction `computeLatestStartDates(Graph graph)` qui résout le problème et qui stockera la solution dans le tableau `latest_start`. Faites un affichage console de la solution.

```
---------------
Print graph:
Vertex 0: (0.316913,0.594309)
Vertex 1: (0.183619,0.159360)
Vertex 2: (0.841916,0.852918)
Vertex 3: (0.644645,0.362065)
Vertex 4: (0.914059,0.759726)
Vertex 5: (0.502944,0.286326)
Vertex 6: (0.745625,0.291710)
Vertex 7: (0.221577,0.727858)
Vertex 8: (0.706567,0.750446)
Vertex 9: (0.473889,0.943460)
Vertex 10: (0.597800,0.442557)
Vertex 11: (0.029429,0.024232)
Vertex 12: (0.769144,0.818098)
Vertex 13: (0.512418,0.683061)
Vertex 14: (0.065862,0.454250)
Vertex 0: 13 -> 7 -> NULL
Vertex 1: NULL
Vertex 2: NULL
Vertex 3: 10 -> NULL
Vertex 4: 12 -> 2 -> NULL
Vertex 5: 10 -> 6 -> 3 -> NULL
Vertex 6: 10 -> 3 -> NULL
Vertex 7: NULL
Vertex 8: 12 -> 4 -> 2 -> NULL
Vertex 9: NULL
Vertex 10: 13 -> NULL
Vertex 11: 1 -> NULL
Vertex 12: 2 -> NULL
Vertex 13: 12 -> 9 -> 8 -> 7 -> NULL
Vertex 14: 0 -> NULL
Topological ordering:
[14 11 5 6 3 10 1 0 13 7 8 4 9 12 2 ]
Print start dates:
vertex 0 : [earliest start date=0.29, latest start date=0.50 ]
vertex 1 : [earliest start date=0.21, latest start date=1.36 ]
vertex 2 : [earliest start date=1.36, latest start date=1.36 ]
vertex 3 : [earliest start date=0.37, latest start date=0.37 ]
vertex 4 : [earliest start date=1.13, latest start date=1.13 ]
vertex 5 : [earliest start date=0.00, latest start date=0.00 ]
vertex 6 : [earliest start date=0.24, latest start date=0.24 ]
vertex 7 : [earliest start date=1.01, latest start date=1.36 ]
vertex 8 : [earliest start date=0.92, latest start date=0.92 ]
vertex 9 : [earliest start date=0.98, latest start date=1.36 ]
vertex 10 : [earliest start date=0.46, latest start date=0.46 ]
vertex 11 : [earliest start date=0.00, latest start date=1.16 ]
vertex 12 : [earliest start date=1.28, latest start date=1.28 ]
vertex 13 : [earliest start date=0.71, latest start date=0.71 ]
vertex 14 : [earliest start date=0.00, latest start date=0.21 ]
```

Un affichage console possible



Un affichage graphique possible : entre crochets
[indice ordre topologique, date au plus tôt, date au plus tard]

TP – Arbres couvrants minimums

L'objectif de ce TP est d'implémenter l'algorithme de Prim vu en cours pour calculer un arbre couvrant minimum (ACM) d'un graphe non-orienté et pondéré (répertoire `spanningtree`). L'algorithme de Prim s'appuie sur la structure de file de priorité qu'il faut au préalable implémenter (répertoire `heap)`.

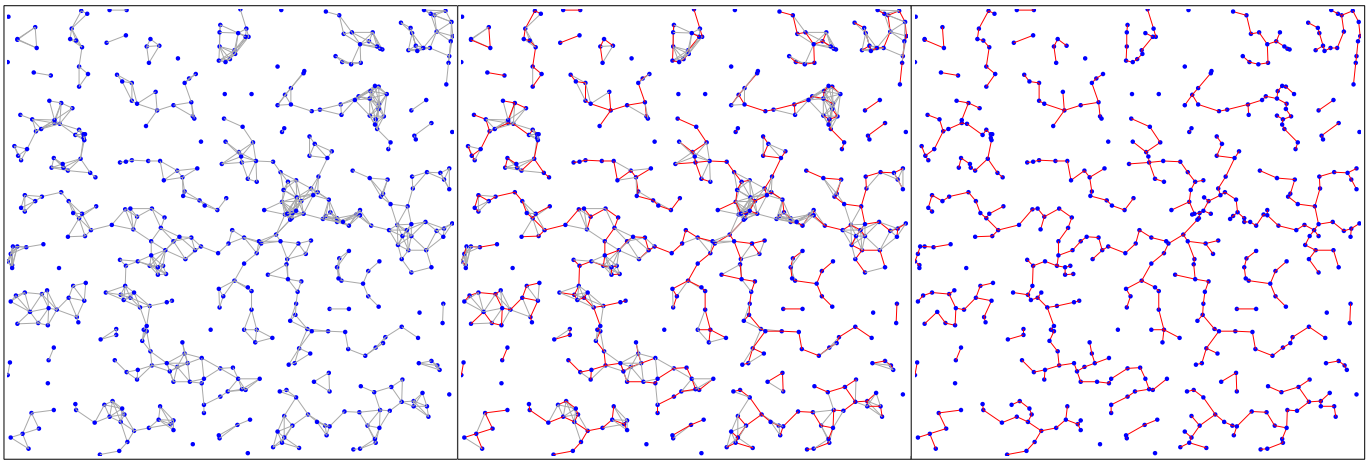
Comme pour les précédents TP, un `makefile` et des fichiers de tests naïfs sont fournis dans le répertoire `spanningtree`. Il suffit de faire `make all` puis ./`testprim` pour tester vos développements.

Algorithme de Prim

L'algorithme de Prim a été présenté en cours. Il s'appuie sur la structure de tas pour construire un arbre couvrant minimum d'un graphe non orienté et pondéré à partir d'un sommet. Si le graphe est composé de plusieurs composantes connexes, ce n'est pas un arbre mais une forêt couvrante minimale qui est calculée.

Nous reprenons la structure de graphe (`Graph`) non-orientée des précédents TPs. Les graphes seront construits à partir de points tirés au hasard dans [0,1]x[0,1]. Les sommets du graphe seront les points tirés au hasard. Une arête {A,B} avec A=(x,y) et B=(x',y') sera présente dans le graphe si la distance entre A et B est plus petite qu'un seuil σ fixé dans [0,1].

A la différence des précédents TPs, le poids d'une arête sera donnée par la distance euclidienne entre le deux sommets de l'arête.

Travail à faire :

- Implémentez une fonction [++] `void Prim(Graph g, int s)` qui calcule une forêt couvrante minimum du graphe `g` avec l'algorithme de Prim en utilisant la représentation par tableau de listes de successeurs. La forêt sera stockée dans le champs `parents` de la structure de graphe.
- Implémentez des tests unitaires dans le fichier `testprim.c`.